

Module 2_CPINN: Causal Physics-Informed Neural Training for Two-Dimensional Electrostatics

Textbook-Style Physics Note for RadiationEffects_proj2

Extending Module 2_PINN with causality-respecting residual weighting across ordered damage, dose, time, or continuation states

Purpose of this note. This document develops the physics and mathematics needed to add a Causal Physics-Informed Neural Network path to Module 2 of `RadiationEffects_proj2`. Module 2 solves electrostatic Poisson physics: radiation-induced charged defects create a space-charge density, the space charge modifies electrostatic potential, and the potential gives electric field. The ordinary Module 2 PINN enforces Poisson's equation at interior collocation points and boundary conditions at boundary points. The causal extension modifies the training objective so that ordered states are learned in a physically meaningful sequence. For Module 2, the ordered variable is not intrinsic electrostatic time, because Poisson's equation is elliptic and quasi-static. Instead, the ordered variable should be a damage time, dose/load step, defect-evolution snapshot, or continuation parameter inherited from the radiation-effects chain.

Contents

Symbols and acronyms used in this document	4
1 Position of Module 2_CPINN in the project	6
2 Recap: Module 2 electrostatic physics	7
2.1 Strong-form Poisson equation	7
2.2 Boundary conditions	8
2.3 Weak form and FEM reference	8
3 Ordinary Module 2 PINN formulation	9
3.1 Neural representation of potential	9
3.2 Poisson residual	9
3.3 Boundary losses	10
3.4 Supervised data loss	10

4	Why causal PINN was introduced	10
4.1	The problem with simultaneous residual minimization	10
4.2	Temporal residual loss by slice	11
4.3	Causal residual weights	12
4.4	Causal weights as an activation front	12
5	Adapting causal PINN to Module 2	13
5.1	Why the adaptation is not automatic	13
5.2	Ordered quasi-static Poisson sequence	14
5.3	Slice-wise Module 2 residual	14
6	Causal Module 2 loss	15
6.1	Prefix loss	15
6.2	Residual-only causal weighting versus full-slice causal weighting	16
6.3	Including the initial or first state	16
7	Nondimensional CPINN formulation	16
7.1	Why nondimensionalization is required	17
7.2	Dimensionless variables	17
7.3	Dimensionless residual and loss	18
8	Optional ordered-consistency residual	18
8.1	Differentiating quasi-static Poisson with respect to the ordered variable	18
8.2	Sensitivity residual for CPINN	19
8.3	Finite-difference alternative	19
9	Architecture choices for Module 2_CPINN	19
9.1	Input-output map	19
9.2	Why local charge alone is insufficient	20
9.3	Activation function	20
9.4	Hard versus soft boundary enforcement	21
10	Sampling strategy	21

10.1 Ordered-state sampling	21
10.2 Interior residual points	22
10.3 Boundary points	22
10.4 Supervised data points	22
11 Training algorithm	22
11.1 Algorithm in mathematical form	22
11.2 Annealing the causality parameter	23
11.3 Causal stopping criterion	23
12 Connection to Module 3 and Module 6	24
12.1 Using Module 3 defect evolution as the causal source	24
12.2 Using Module 6 coupling iteration as the ordered variable	24
12.3 Recommended project sequence	25
13 Diagnostics and validation	25
13.1 Potential error	25
13.2 Electric-field error	25
13.3 Residual-by-slice plot	25
13.4 Causal-weight plot	26
13.5 Gauss-law consistency	26
14 Recommended first CPINN test cases	26
14.1 Case 1: zero charge with linear potential	26
14.2 Case 2: Gaussian charge amplitude continuation	27
14.3 Case 3: defect snapshots from Module 3	27
14.4 Case 4: coupled-state snapshots from Module 6	27
15 Common failure modes	27
15.1 Mistaking CPINN for a new electrostatic model	27
15.2 Using unscaled losses in causal exponentials	28
15.3 Overweighting supervised data	28

15.4 Using too many ordered slices too early	28
15.5 Poor charge-parameter representation	28
16 What should be implemented first	28
17 Minimal mathematical specification for the code	29
17.1 Inputs	29
17.2 Residual	29
17.3 Slice losses	29
17.4 Causal weights	30
17.5 Total loss	30
18 Summary	30

Symbols and acronyms used in this document

Symbol / acronym	Meaning	Typical units
CPINN	Causal Physics-Informed Neural Network in this note. This is not the same as conservative PINN, which is also sometimes abbreviated cPINN in the literature.	–
PINN	Physics-informed neural network	–
PDE	Partial differential equation	–
FEM	Finite-element method	–
AD	Automatic differentiation	–
MLP	Multilayer perceptron	–
Ω	Two-dimensional Module 2 computational domain	m ²
$\partial\Omega$	Boundary of the domain	m
Γ_D	Dirichlet boundary segment	m
Γ_N	Neumann boundary segment	m
$\mathbf{x} = (x, y)$	Spatial coordinate	m

Symbol / acronym	Meaning	Typical units
τ	Ordered causal/continuation variable: damage time, dose step, load factor, or upstream defect-evolution snapshot index	s, Gy, fluence, or dimensionless
μ	Parameter vector: charge-shape parameters, material parameters, boundary voltages, geometry parameters	various
$\phi(\mathbf{x}; \tau, \mu)$	Exact quasi-static electrostatic potential at ordered state τ	V
$\phi_\theta(\mathbf{x}, \tau, \mu)$	Neural-network approximation to potential	V or dimensionless
θ	Trainable neural-network weights and biases	—
$\mathbf{E}$	Electric field, $\mathbf{E} = -\nabla\phi$	V/m
ρ	Total space-charge density	C/m ³
ρ_{def}	Defect-induced space-charge density	C/m ³
$\varepsilon, \varepsilon_{\text{Si}}$	Permittivity, silicon permittivity	F/m
q	Elementary charge magnitude	C
n, p	Electron and hole concentrations	m ⁻³
N_D^+, N_A^-	Ionized donor and acceptor concentrations	m ⁻³
C_i	Concentration of defect species i	m ⁻³
z_i	Effective charge number of defect species i	—
$\mathcal{R}_{\text{P}}$	Poisson residual	scaled or C/m ³
$\mathcal{R}_{\text{C}}$	Optional sensitivity/ordered-consistency residual	scaled
$\mathcal{L}_{\text{data}}$	Supervised mismatch loss against FEM/reference values	scaled
$\mathcal{L}_{\text{PDE}}$	Interior PDE residual loss	scaled
$\mathcal{L}_{\text{D}}$	Dirichlet boundary loss	scaled
$\mathcal{L}_{\text{N}}$	Neumann boundary loss	scaled
$\mathcal{L}_{\text{BC}}$	Total boundary-condition loss	scaled
$\mathcal{L}_{\text{slice}, i}$	Loss contribution at ordered state τ_i	scaled
w_i	Causal residual weight for ordered state τ_i	—
ϵ_c	Causality parameter controlling how sharply earlier errors suppress later residuals	—
N_τ	Number of ordered states/slices	—

Symbol / acronym	Meaning	Typical units
N_r	Number of interior residual/collocation points per slice	–
N_D^{bc}	Number of Dirichlet boundary points per slice	–
N_N^{bc}	Number of Neumann boundary points per slice	–
N_s	Number of supervised reference points per slice	–
L	Characteristic length scale	m
ϕ_0	Characteristic potential scale	V
ρ_0	Characteristic charge-density scale	C/m ³
$\hat{x}, \hat{y}, \hat{\tau}, \hat{\phi}, \hat{\rho}$	Dimensionless coordinate, ordered variable, potential, and charge density	–

1 Position of Module 2_CPINN in the project

The baseline Module 2 electrostatic solver maps a charge distribution into potential and electric field:

$$\rho(x, y) \longrightarrow \phi(x, y) \longrightarrow \mathbf{E}(x, y). \quad (1.1)$$

The ordinary Module 2 PINN note introduced a neural surrogate for this mapping by combining supervised FEM data with the strong-form Poisson residual and boundary-condition penalties. The present document keeps that same physical model but changes the *training logic*. The CPINN objective is designed to learn ordered states progressively rather than allowing the optimizer to reduce residuals everywhere at once.

The intended Module 2 path is therefore

$$\text{Module 2 FEM} \longleftrightarrow \text{Module 2_PINN} \longrightarrow \text{Module 2_CPINN}. \quad (1.2)$$

The FEM solver remains the reference implementation. The ordinary PINN remains the simpler learning baseline. The CPINN is a training strategy to test whether causality-respecting weighting improves robustness when the electrostatic problems are arranged along a meaningful ordered coordinate such as irradiation time, accumulated dose, defect-amplitude continuation, or coupled-iteration index.

Central caveat. Poisson electrostatics is not a time-evolution equation. A voltage or charge distribution at one time does not evolve to the next time through Module 2 alone. Therefore, a causal PINN applied to a single static Poisson solve is not physically necessary. The causal idea becomes meaningful for Module 2 only when the Poisson solves are embedded in an ordered radiation-effects sequence, for example $\rho(x, y, t_0), \rho(x, y, t_1), \dots$ generated by defect evolution or by accumulated dose.

This point is not a minor detail. The original causal PINN idea was developed for time-dependent systems where the solution at later time depends on the correctness of earlier time. Module 2 is elliptic and quasi-static. Therefore, the right interpretation for this project is:

Module 2_CPINN is a causality-respecting ordered training curriculum for a sequence of quasi-static Poisson problems. (1.3)

It should not be described as a new electrostatic law.

2 Recap: Module 2 electrostatic physics

2.1 Strong-form Poisson equation

Module 2 solves Poisson's equation in a two-dimensional silicon cross section. The dimensional equation is

$$\nabla \cdot (\varepsilon \nabla \phi) = -\rho, \quad (2.1)$$

where ε is permittivity, ϕ is electrostatic potential, and ρ is total space-charge density. For uniform silicon permittivity,

$$\varepsilon_{\text{Si}} \nabla^2 \phi = -\rho, \quad (2.2)$$

with

$$\nabla^2 \phi = \frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2}. \quad (2.3)$$

The electric field is

$$\mathbf{E} = -\nabla \phi = - \begin{bmatrix} \partial \phi / \partial x \\ \partial \phi / \partial y \end{bmatrix}. \quad (2.4)$$

The total charge density can be represented as

$$\rho = q \left(p - n + N_D^+ - N_A^- \right) + \rho_{\text{def}}, \quad (2.5)$$

where p and n are hole and electron concentrations, N_D^+ and N_A^- are ionized dopant concentrations,

and ρ_{def} is charge due to radiation-induced defects. For a set of charged defect species,

$$\rho_{\text{def}} = q \sum_{j=1}^M z_j C_j, \quad (2.6)$$

where C_j is the concentration of defect species j and z_j is its effective charge number.

2.2 Boundary conditions

On a Dirichlet boundary Γ_D , the potential is prescribed:

$$\phi(\mathbf{x}) = \phi_D(\mathbf{x}), \quad \mathbf{x} \in \Gamma_D. \quad (2.7)$$

On a Neumann boundary Γ_N , the normal flux is prescribed:

$$\mathbf{n} \cdot \varepsilon \nabla \phi(\mathbf{x}) = g_N(\mathbf{x}), \quad \mathbf{x} \in \Gamma_N. \quad (2.8)$$

Because $\mathbf{E} = -\nabla \phi$, the outward electric displacement is $\mathbf{D} \cdot \mathbf{n} = -\mathbf{n} \cdot \varepsilon \nabla \phi$. Thus the sign convention for g_N must be kept explicit.

For a pure Neumann problem, the potential is determined only up to an additive constant. In that case, a gauge condition such as

$$\int_{\Omega} \phi d\Omega = 0 \quad (2.9)$$

or one fixed reference node must be added. Most first Module 2 cases should include at least one Dirichlet boundary to avoid this ambiguity.

2.3 Weak form and FEM reference

The FEM reference solver comes from the weak form. Multiply Eq. (2.1) by a test function v and integrate over Ω :

$$\int_{\Omega} v \nabla \cdot (\varepsilon \nabla \phi) d\Omega = - \int_{\Omega} v \rho d\Omega. \quad (2.10)$$

Using integration by parts,

$$\int_{\Omega} v \nabla \cdot (\varepsilon \nabla \phi) d\Omega = \int_{\partial\Omega} v \mathbf{n} \cdot \varepsilon \nabla \phi d\Gamma - \int_{\Omega} \nabla v \cdot \varepsilon \nabla \phi d\Omega. \quad (2.11)$$

Therefore,

$$\int_{\Omega} \nabla v \cdot \varepsilon \nabla \phi d\Omega = \int_{\Omega} v \rho d\Omega + \int_{\partial\Omega} v \mathbf{n} \cdot \varepsilon \nabla \phi d\Gamma. \quad (2.12)$$

When Dirichlet conditions are imposed strongly and Neumann data are given on Γ_N ,

$$\int_{\Omega} \nabla v \cdot \varepsilon \nabla \phi d\Omega = \int_{\Omega} v \rho d\Omega + \int_{\Gamma_N} v g_N d\Gamma. \quad (2.13)$$

This weak form leads to the discrete FEM system

$$K\Phi = b, \quad (2.14)$$

where Φ contains nodal potential values. The CPINN does not replace this derivation. It uses FEM solutions as reference data and uses the strong-form residual as an additional training constraint.

3 Ordinary Module 2 PINN formulation

3.1 Neural representation of potential

The Module 2 PINN represents the potential by a neural network

$$\phi_\theta = \mathcal{N}_\theta(x, y, \boldsymbol{\mu}), \quad (3.1)$$

where $\boldsymbol{\mu}$ is a parameter vector describing the electrostatic case. Examples include charge amplitude, charge location, Gaussian widths, boundary voltages, permittivity, and geometry dimensions.

The electric field predicted by the network is not a separately trained quantity unless explicitly chosen. It is naturally obtained by differentiation:

$$\mathbf{E}_\theta(x, y; \boldsymbol{\mu}) = -\nabla \phi_\theta(x, y; \boldsymbol{\mu}). \quad (3.2)$$

3.2 Poisson residual

For constant permittivity, define the strong-form residual

$$\mathcal{R}_P(x, y; \boldsymbol{\mu}, \boldsymbol{\theta}) = \varepsilon_{\text{Si}} \left(\frac{\partial^2 \phi_\theta}{\partial x^2} + \frac{\partial^2 \phi_\theta}{\partial y^2} \right) + \rho(x, y; \boldsymbol{\mu}). \quad (3.3)$$

The exact solution satisfies $\mathcal{R}_P = 0$ at every interior point. A standard PINN evaluates this residual at collocation points $\{\mathbf{x}_r^{(k)}\}_{k=1}^{N_r}$:

$$\mathcal{L}_{\text{PDE}}(\boldsymbol{\theta}) = \frac{1}{N_r} \sum_{k=1}^{N_r} \left| \mathcal{R}_P(\mathbf{x}_r^{(k)}; \boldsymbol{\mu}^{(k)}, \boldsymbol{\theta}) \right|^2. \quad (3.4)$$

For spatially variable permittivity,

$$\mathcal{R}_P = \nabla \cdot (\varepsilon \nabla \phi_\theta) + \rho = \varepsilon \nabla^2 \phi_\theta + \nabla \varepsilon \cdot \nabla \phi_\theta + \rho. \quad (3.5)$$

The first implementation should use constant ε_{Si} unless there is a clear need for material interfaces.

3.3 Boundary losses

The Dirichlet boundary loss is

$$\mathcal{L}_D(\boldsymbol{\theta}) = \frac{1}{N_D} \sum_{k=1}^{N_D} \left| \phi_{\theta}(\mathbf{x}_D^{(k)}; \boldsymbol{\mu}^{(k)}) - \phi_D(\mathbf{x}_D^{(k)}; \boldsymbol{\mu}^{(k)}) \right|^2. \quad (3.6)$$

The Neumann boundary loss is

$$\mathcal{L}_N(\boldsymbol{\theta}) = \frac{1}{N_N} \sum_{k=1}^{N_N} \left| \mathbf{n}^{(k)} \cdot \varepsilon \nabla \phi_{\theta}(\mathbf{x}_N^{(k)}; \boldsymbol{\mu}^{(k)}) - g_N(\mathbf{x}_N^{(k)}; \boldsymbol{\mu}^{(k)}) \right|^2. \quad (3.7)$$

The total boundary loss is

$$\mathcal{L}_{BC} = \lambda_D \mathcal{L}_D + \lambda_N \mathcal{L}_N. \quad (3.8)$$

3.4 Supervised data loss

If the FEM solver provides reference values

$$\left\{ \left(x_s^{(k)}, y_s^{(k)}, \boldsymbol{\mu}^{(k)}, \phi_{\text{FEM}}^{(k)} \right) \right\}_{k=1}^{N_s}, \quad (3.9)$$

then the supervised loss is

$$\mathcal{L}_{\text{data}}(\boldsymbol{\theta}) = \frac{1}{N_s} \sum_{k=1}^{N_s} \left| \phi_{\theta}(x_s^{(k)}, y_s^{(k)}; \boldsymbol{\mu}^{(k)}) - \phi_{\text{FEM}}^{(k)} \right|^2. \quad (3.10)$$

The ordinary PINN objective is

$$\mathcal{L}_{\text{total}}(\boldsymbol{\theta}) = \lambda_s \mathcal{L}_{\text{data}} + \lambda_r \mathcal{L}_{\text{PDE}} + \lambda_D \mathcal{L}_D + \lambda_N \mathcal{L}_N + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}. \quad (3.11)$$

4 Why causal PINN was introduced

4.1 The problem with simultaneous residual minimization

Consider a time-dependent PDE in abstract form:

$$\frac{\partial u}{\partial t} = \mathcal{F}[u], \quad (x, t) \in \Omega \times [0, T], \quad (4.1)$$

with initial condition

$$u(x, 0) = u_0(x). \quad (4.2)$$

A standard continuous-time PINN represents

$$u_\theta = \mathcal{N}_\theta(x, t) \quad (4.3)$$

and defines the residual

$$\mathcal{R}(x, t; \theta) = \frac{\partial u_\theta}{\partial t}(x, t) - \mathcal{F}[u_\theta](x, t). \quad (4.4)$$

The ordinary residual loss is

$$\mathcal{L}_r = \frac{1}{N_t N_x} \sum_{i=1}^{N_t} \sum_{j=1}^{N_x} |\mathcal{R}(x_j, t_i; \theta)|^2. \quad (4.5)$$

This loss treats all time slices simultaneously. It does not explicitly require the network to first learn early times before later times.

To see the issue, discretize the time derivative by a forward or backward difference. For clarity, use

$$\frac{\partial u}{\partial t}(x, t_i) \approx \frac{u(x, t_i) - u(x, t_{i-1})}{\Delta t}. \quad (4.6)$$

Then the residual at t_i takes the schematic form

$$\mathcal{R}_i(x; \theta) \approx \frac{u_\theta(x, t_i) - u_\theta(x, t_{i-1})}{\Delta t} - \mathcal{F}[u_\theta](x, t_{i-1}). \quad (4.7)$$

The residual at t_i depends on the network predictions at earlier time t_{i-1} . Therefore, a small residual at t_i is physically meaningful only if the prediction at t_{i-1} is already accurate. Otherwise the model may produce a later-time output that is consistent with a wrong earlier-time output.

This is the core causal-PINN diagnosis: ordinary PINNs can reduce residuals at later times before early-time physics has been learned correctly. That violates the causal structure of evolution equations.

4.2 Temporal residual loss by slice

Define the residual loss at a single time slice t_i as

$$\mathcal{L}_{r,i}(\theta) = \frac{1}{N_x} \sum_{j=1}^{N_x} |\mathcal{R}(x_j, t_i; \theta)|^2. \quad (4.8)$$

Then the ordinary residual loss is

$$\mathcal{L}_r(\theta) = \frac{1}{N_t} \sum_{i=1}^{N_t} \mathcal{L}_{r,i}(\theta). \quad (4.9)$$

The ordinary sum gives every time slice equal access to the optimizer from the beginning. A causal training strategy instead assigns time-dependent weights w_i so that early slices dominate until their residuals are small.

4.3 Causal residual weights

The causal residual loss is

$$\mathcal{L}_r^{\text{causal}}(\theta) = \frac{1}{N_t} \sum_{i=1}^{N_t} w_i \mathcal{L}_{r,i}(\theta). \quad (4.10)$$

The defining choice is

$$w_i = \exp \left[-\epsilon_c \sum_{k=1}^{i-1} \mathcal{L}_{r,k}(\theta) \right], \quad w_1 = 1. \quad (4.11)$$

The parameter $\epsilon_c > 0$ controls how strongly earlier residuals suppress later residuals. If the earlier losses are large, then w_i is small, and the optimizer effectively ignores later time slices. As the earlier losses decrease, w_i increases toward one, activating later slices.

In implementation, the weights are usually treated as non-differentiated weights:

$$w_i = \exp \left[-\epsilon_c \text{stopgrad} \left(\sum_{k=1}^{i-1} \mathcal{L}_{r,k}(\theta) \right) \right]. \quad (4.12)$$

The stop-gradient operation means the optimizer does not exploit the derivative of the weight formula itself. The weights act as a training curriculum, not as another differentiable path that can be gamed by the optimizer.

Interpretation. A causal PINN does not add a new physical term to the PDE. It changes how the PDE residual loss is weighted during training. Earlier states must become accurate before later states receive full optimization weight.

4.4 Causal weights as an activation front

The causal weights create an activation front moving from early to late time. At the beginning, one often sees

$$w_1 \approx 1, \quad w_2, w_3, \dots, w_{N_t} \ll 1. \quad (4.13)$$

After early residuals decrease,

$$w_2, w_3, \dots \quad (4.14)$$

become larger. Near convergence,

$$w_i \approx 1 \quad \text{for all } i. \quad (4.15)$$

A practical convergence diagnostic is therefore

$$\min_i w_i > 1 - \delta_w, \quad (4.16)$$

where δ_w is a tolerance such as 10^{-2} or 10^{-3} after losses have been nondimensionalized.

5 Adapting causal PINN to Module 2

5.1 Why the adaptation is not automatic

Module 2 solves

$$\nabla \cdot (\varepsilon \nabla \phi) = -\rho. \quad (5.1)$$

There is no $\partial\phi/\partial t$ term. This means the Poisson solution at state τ_i is obtained from the charge distribution at the same state:

$$\rho(\mathbf{x}; \tau_i) \longrightarrow \phi(\mathbf{x}; \tau_i). \quad (5.2)$$

The Poisson equation itself does not state

$$\phi(\mathbf{x}; \tau_i) = \text{time evolution of } \phi(\mathbf{x}; \tau_{i-1}). \quad (5.3)$$

Thus, a causal formulation for Module 2 must be attached to an ordered sequence supplied by the larger radiation-effects problem:

$$\rho(\mathbf{x}; \tau_0), \rho(\mathbf{x}; \tau_1), \dots, \rho(\mathbf{x}; \tau_{N_\tau}). \quad (5.4)$$

Here τ may represent:

- physical defect-evolution time from Module 3;
- accumulated radiation dose or fluence from Module 1 to Module 3;
- an artificial continuation factor multiplying defect charge;
- a boundary-voltage ramp;
- a multiphysics coupling iteration index in Module 6.

The strongest physical choice is defect-evolution time or dose, because those are genuine ordered variables in the project. The safest first implementation choice is an artificial defect-amplitude continuation parameter, because it is easy to generate and easy to debug.

5.2 Ordered quasi-static Poisson sequence

Define an ordered variable

$$\tau_0 < \tau_1 < \dots < \tau_{N_\tau}. \quad (5.5)$$

At each state, Module 2 satisfies

$$\nabla \cdot (\varepsilon \nabla \phi(\mathbf{x}; \tau_i, \boldsymbol{\mu})) = -\rho(\mathbf{x}; \tau_i, \boldsymbol{\mu}), \quad \mathbf{x} \in \Omega. \quad (5.6)$$

The boundary conditions may also depend on τ_i :

$$\phi(\mathbf{x}; \tau_i, \boldsymbol{\mu}) = \phi_D(\mathbf{x}; \tau_i, \boldsymbol{\mu}), \quad \mathbf{x} \in \Gamma_D, \quad (5.7)$$

$$\mathbf{n} \cdot \varepsilon \nabla \phi(\mathbf{x}; \tau_i, \boldsymbol{\mu}) = g_N(\mathbf{x}; \tau_i, \boldsymbol{\mu}), \quad \mathbf{x} \in \Gamma_N. \quad (5.8)$$

The CPINN representation is

$$\phi_\theta = \mathcal{N}_\theta(x, y, \tau, \boldsymbol{\mu}). \quad (5.9)$$

This network learns a family of electrostatic solutions indexed by τ .

5.3 Slice-wise Module 2 residual

For each ordered state τ_i , define the residual

$$\mathcal{R}_P^{(i)}(\mathbf{x}; \boldsymbol{\theta}) = \nabla \cdot (\varepsilon \nabla \phi_\theta(\mathbf{x}, \tau_i, \boldsymbol{\mu})) + \rho(\mathbf{x}; \tau_i, \boldsymbol{\mu}). \quad (5.10)$$

For constant ε_{Si} ,

$$\mathcal{R}_P^{(i)}(\mathbf{x}; \boldsymbol{\theta}) = \varepsilon_{Si} \left(\frac{\partial^2 \phi_\theta}{\partial x^2}(\mathbf{x}, \tau_i, \boldsymbol{\mu}) + \frac{\partial^2 \phi_\theta}{\partial y^2}(\mathbf{x}, \tau_i, \boldsymbol{\mu}) \right) + \rho(\mathbf{x}; \tau_i, \boldsymbol{\mu}). \quad (5.11)$$

The residual loss for the slice is

$$\mathcal{L}_{PDE,i}(\boldsymbol{\theta}) = \frac{1}{N_r} \sum_{j=1}^{N_r} \left| \mathcal{R}_P^{(i)}(\mathbf{x}_{r,j}^{(i)}; \boldsymbol{\theta}) \right|^2. \quad (5.12)$$

Similarly,

$$\mathcal{L}_{D,i}(\boldsymbol{\theta}) = \frac{1}{N_D} \sum_{j=1}^{N_D} \left| \phi_\theta(\mathbf{x}_{D,j}^{(i)}, \tau_i, \boldsymbol{\mu}) - \phi_D(\mathbf{x}_{D,j}^{(i)}, \tau_i, \boldsymbol{\mu}) \right|^2, \quad (5.13)$$

$$\mathcal{L}_{N,i}(\boldsymbol{\theta}) = \frac{1}{N_N} \sum_{j=1}^{N_N} \left| \mathbf{n}_j \cdot \varepsilon \nabla \phi_\theta(\mathbf{x}_{N,j}^{(i)}, \tau_i, \boldsymbol{\mu}) - g_N(\mathbf{x}_{N,j}^{(i)}, \tau_i, \boldsymbol{\mu}) \right|^2, \quad (5.14)$$

and, when FEM data are available,

$$\mathcal{L}_{\text{data},i}(\boldsymbol{\theta}) = \frac{1}{N_s} \sum_{j=1}^{N_s} \left| \phi_{\boldsymbol{\theta}}(\mathbf{x}_{s,j}^{(i)}, \tau_i, \boldsymbol{\mu}) - \phi_{\text{FEM}}(\mathbf{x}_{s,j}^{(i)}, \tau_i, \boldsymbol{\mu}) \right|^2. \quad (5.15)$$

Define the total slice loss

$$\mathcal{L}_{\text{slice},i}(\boldsymbol{\theta}) = \lambda_r \mathcal{L}_{\text{PDE},i} + \lambda_D \mathcal{L}_{D,i} + \lambda_N \mathcal{L}_{N,i} + \lambda_s \mathcal{L}_{\text{data},i}. \quad (5.16)$$

This is the loss that would be minimized for state τ_i in an ordinary PINN. CPINN changes how the slice losses are weighted.

6 Causal Module 2 loss

6.1 Prefix loss

The causal logic says that state τ_i should receive full training weight only if all previous states $\tau_0, \dots, \tau_{i-1}$ are already reasonably accurate. Define the prefix loss before state i as

$$S_{i-1}(\boldsymbol{\theta}) = \sum_{k=0}^{i-1} \mathcal{L}_{\text{slice},k}(\boldsymbol{\theta}), \quad S_{-1} = 0. \quad (6.1)$$

Then define the causal weight

$$w_i = \exp[-\epsilon_c \text{stopgrad}(S_{i-1}(\boldsymbol{\theta}))]. \quad (6.2)$$

The first state has

$$w_0 = \exp(0) = 1. \quad (6.3)$$

The second state has

$$w_1 = \exp[-\epsilon_c \text{stopgrad}(\mathcal{L}_{\text{slice},0})]. \quad (6.4)$$

If the first state is poorly learned, w_1 is small. As $\mathcal{L}_{\text{slice},0}$ decreases, w_1 grows.

The full Module 2 CPINN loss is

$$\mathcal{L}_{\text{total}}^{\text{CPINN}}(\boldsymbol{\theta}) = \frac{1}{N_{\tau} + 1} \sum_{i=0}^{N_{\tau}} w_i \mathcal{L}_{\text{slice},i}(\boldsymbol{\theta}) + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}(\boldsymbol{\theta}). \quad (6.5)$$

This is the central equation for the planned Module 2_CPINN extension.

6.2 Residual-only causal weighting versus full-slice causal weighting

There are two reasonable versions.

Version A: residual-only causal weighting. Only the PDE residual is causally weighted:

$$\mathcal{L}_{\text{total}}^A = \frac{1}{N_\tau + 1} \sum_{i=0}^{N_\tau} [w_i \lambda_r \mathcal{L}_{\text{PDE},i} + \lambda_D \mathcal{L}_{D,i} + \lambda_N \mathcal{L}_{N,i} + \lambda_s \mathcal{L}_{\text{data},i}] + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}. \quad (6.6)$$

This is closest to the original causal-PINN idea, which focused on residual losses.

Version B: full-slice causal weighting. The entire slice loss is causally weighted:

$$\mathcal{L}_{\text{total}}^B = \frac{1}{N_\tau + 1} \sum_{i=0}^{N_\tau} w_i [\lambda_r \mathcal{L}_{\text{PDE},i} + \lambda_D \mathcal{L}_{D,i} + \lambda_N \mathcal{L}_{N,i} + \lambda_s \mathcal{L}_{\text{data},i}] + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}. \quad (6.7)$$

This is more aggressive: later states are withheld almost completely until earlier states are learned.

For Module 2, Version B is easier to reason about during early development because one weight controls the whole ordered slice. Version A is closer to the literature and may be preferable once debugging is complete.

6.3 Including the initial or first state

For an evolution equation, the initial condition is the first cause. For Module 2, the first ordered state often plays the analogous role. If τ_0 is a zero-dose or zero-defect state, it should be learned first. If $\rho(\mathbf{x}; \tau_0) = 0$ and boundary conditions are simple, it may even have an analytical solution. This makes it an ideal anchor.

For example, with zero charge and left/right Dirichlet voltages,

$$\rho = 0, \quad \phi(0, y) = V_L, \quad \phi(L_x, y) = V_R, \quad \frac{\partial \phi}{\partial y}(x, 0) = \frac{\partial \phi}{\partial y}(x, L_y) = 0, \quad (6.8)$$

the exact solution is

$$\phi(x, y) = V_L + \frac{V_R - V_L}{L_x} x. \quad (6.9)$$

This anchor state is a natural first slice for Module 2_CPINN.

7 Nondimensional CPINN formulation

7.1 Why nondimensionalization is required

The dimensional residual in Eq. (5.11) has units of C/m³. Its magnitude can be extremely different from a voltage data loss. Causal weights contain exponentials of cumulative losses:

$$w_i = \exp[-\epsilon_c S_{i-1}]. \quad (7.1)$$

If S_{i-1} is dimensional or numerically huge, the weights collapse to zero. If it is too small, the weights stay near one and the causal mechanism disappears. Therefore, every slice loss entering the causal weight must be dimensionless and numerically controlled.

Implementation rule. Do not use raw dimensional Poisson residuals inside the exponential causal weights. Always nondimensionalize or normalize each slice loss before forming w_i .

7.2 Dimensionless variables

Choose scales

$$\hat{x} = \frac{x}{L}, \quad \hat{y} = \frac{y}{L}, \quad \hat{\phi} = \frac{\phi}{\phi_0}, \quad \hat{\rho} = \frac{\rho}{\rho_0}. \quad (7.2)$$

For the ordered variable, choose

$$\hat{\tau} = \frac{\tau - \tau_{\min}}{\tau_{\max} - \tau_{\min}}. \quad (7.3)$$

For uniform permittivity,

$$\epsilon_{\text{Si}} \frac{\phi_0}{L^2} \left(\frac{\partial^2 \hat{\phi}}{\partial \hat{x}^2} + \frac{\partial^2 \hat{\phi}}{\partial \hat{y}^2} \right) = -\rho_0 \hat{\rho}. \quad (7.4)$$

Thus

$$\frac{\partial^2 \hat{\phi}}{\partial \hat{x}^2} + \frac{\partial^2 \hat{\phi}}{\partial \hat{y}^2} + \beta \hat{\rho} = 0, \quad (7.5)$$

where

$$\beta = \frac{\rho_0 L^2}{\epsilon_{\text{Si}} \phi_0}. \quad (7.6)$$

A convenient choice is

$$\phi_0 = \frac{\rho_0 L^2}{\epsilon_{\text{Si}}}, \quad (7.7)$$

which gives $\beta = 1$. If boundary voltages dominate, another reasonable choice is

$$\phi_0 = \max \left(|V_L|, |V_R|, \frac{\rho_0 L^2}{\epsilon_{\text{Si}}} \right). \quad (7.8)$$

Then β is retained explicitly.

7.3 Dimensionless residual and loss

The dimensionless residual at slice i is

$$\hat{\mathcal{R}}_P^{(i)} = \frac{\partial^2 \hat{\phi}_\theta}{\partial \hat{x}^2} + \frac{\partial^2 \hat{\phi}_\theta}{\partial \hat{y}^2} + \beta \hat{\rho}(\hat{x}, \hat{y}; \hat{\tau}_i, \boldsymbol{\mu}). \quad (7.9)$$

The corresponding loss is

$$\hat{\mathcal{L}}_{\text{PDE},i} = \frac{1}{N_r} \sum_{j=1}^{N_r} \left| \hat{\mathcal{R}}_P^{(i)}(\hat{\mathbf{x}}_{r,j}) \right|^2. \quad (7.10)$$

Use dimensionless versions of $\mathcal{L}_{D,i}$, $\mathcal{L}_{N,i}$, and $\mathcal{L}_{\text{data},i}$ as well. The prefix loss used in the causal weights should be

$$\hat{S}_{i-1} = \sum_{k=0}^{i-1} \hat{\mathcal{L}}_{\text{slice},k}, \quad (7.11)$$

not the dimensional version.

8 Optional ordered-consistency residual

8.1 Differentiating quasi-static Poisson with respect to the ordered variable

Although Module 2 is not dynamic, one can derive a useful consistency equation if ρ changes smoothly with τ . Start with

$$\nabla \cdot (\varepsilon \nabla \phi(\mathbf{x}, \tau)) + \rho(\mathbf{x}, \tau) = 0. \quad (8.1)$$

Differentiate both sides with respect to τ :

$$\frac{\partial}{\partial \tau} [\nabla \cdot (\varepsilon \nabla \phi)] + \frac{\partial \rho}{\partial \tau} = 0. \quad (8.2)$$

If ε does not depend on τ ,

$$\nabla \cdot \left(\varepsilon \nabla \frac{\partial \phi}{\partial \tau} \right) + \frac{\partial \rho}{\partial \tau} = 0. \quad (8.3)$$

For constant ε_{Si} ,

$$\varepsilon_{\text{Si}} \nabla^2 \left(\frac{\partial \phi}{\partial \tau} \right) + \frac{\partial \rho}{\partial \tau} = 0. \quad (8.4)$$

This is not a time-evolution equation. It is a sensitivity equation. It says that the rate of change of potential with respect to dose/time/load factor is governed by the rate of change of charge density.

8.2 Sensitivity residual for CPINN

For the neural network, define

$$\mathcal{R}_C(\mathbf{x}, \tau; \boldsymbol{\theta}) = \nabla \cdot \left(\varepsilon \nabla \frac{\partial \phi_\theta}{\partial \tau} \right) + \frac{\partial \rho}{\partial \tau}. \quad (8.5)$$

For constant permittivity,

$$\mathcal{R}_C = \varepsilon_{\text{Si}} \left(\frac{\partial^3 \phi_\theta}{\partial x^2 \partial \tau} + \frac{\partial^3 \phi_\theta}{\partial y^2 \partial \tau} \right) + \frac{\partial \rho}{\partial \tau}. \quad (8.6)$$

A corresponding loss is

$$\mathcal{L}_{\text{sens},i} = \frac{1}{N_r} \sum_{j=1}^{N_r} \left| \mathcal{R}_C(\mathbf{x}_{r,j}^{(i)}, \tau_i; \boldsymbol{\theta}) \right|^2. \quad (8.7)$$

This residual can improve smoothness across ordered states, but it requires third derivatives of the network output if implemented in strong form. That can be expensive and numerically delicate in MATLAB. Therefore, this term should be optional, not part of the first CPINN implementation.

8.3 Finite-difference alternative

A more practical alternative is to use a finite-difference consistency penalty between adjacent ordered states. Suppose the network predicts $\phi_\theta(\mathbf{x}, \tau_i)$ and $\phi_\theta(\mathbf{x}, \tau_{i-1})$. Define

$$\partial_\tau^\Delta \phi_\theta(\mathbf{x}, \tau_i) = \frac{\phi_\theta(\mathbf{x}, \tau_i) - \phi_\theta(\mathbf{x}, \tau_{i-1})}{\tau_i - \tau_{i-1}}, \quad (8.8)$$

and

$$\partial_\tau^\Delta \rho(\mathbf{x}, \tau_i) = \frac{\rho(\mathbf{x}, \tau_i) - \rho(\mathbf{x}, \tau_{i-1})}{\tau_i - \tau_{i-1}}. \quad (8.9)$$

Then enforce

$$\nabla \cdot \left(\varepsilon \nabla \partial_\tau^\Delta \phi_\theta \right) + \partial_\tau^\Delta \rho \approx 0. \quad (8.10)$$

This still requires second spatial derivatives, but avoids explicit third mixed derivatives from automatic differentiation. It is a better optional add-on for MATLAB.

9 Architecture choices for Module 2_CPINN

9.1 Input-output map

The recommended first architecture is a scalar-output MLP:

$$(\hat{x}, \hat{y}, \hat{\tau}, \hat{\boldsymbol{\mu}}) \mapsto \hat{\phi}_\theta. \quad (9.1)$$

The electric field is then obtained from derivatives:

$$\hat{E}_{x,\theta} = -\frac{\partial \hat{\phi}_\theta}{\partial \hat{x}}, \quad \hat{E}_{y,\theta} = -\frac{\partial \hat{\phi}_\theta}{\partial \hat{y}}. \quad (9.2)$$

Dimensional fields are recovered by

$$E_x = \frac{\phi_0}{L} \hat{E}_x, \quad E_y = \frac{\phi_0}{L} \hat{E}_y. \quad (9.3)$$

9.2 Why local charge alone is insufficient

A tempting input choice is

$$(x, y, \rho(x, y)) \mapsto \phi(x, y). \quad (9.4)$$

This is generally insufficient. Poisson's equation is nonlocal: the potential at one point depends on the charge distribution and boundary conditions over the entire domain. Two different global charge distributions can have the same local value $\rho(x, y)$ at one point but produce different potentials there.

Therefore, the network input must encode the global case. There are three practical options:

1. **Analytic parameter encoding.** If the charge is generated by a small number of parameters, feed those parameters into the network. Example: Gaussian amplitude, center, widths, and background charge.
2. **Field encoder.** If the charge distribution is a full grid, use a separate encoder network to compress $\rho(x, y)$ into latent variables. This is more advanced and closer to operator learning.
3. **Single-case PINN.** Train one CPINN for one ordered sequence of charge maps. Then the network input can be only (x, y, τ) because the sequence itself is fixed.

For the first MATLAB implementation, use either option 1 or option 3. Avoid a full field encoder until the basic CPINN loss is tested.

9.3 Activation function

The strong-form Poisson residual requires second derivatives of the network output. Therefore, avoid ReLU for the main PINN because it has zero second derivative almost everywhere. Use smooth activations such as

$$\tanh(z), \quad \sin(z), \quad \text{softplus}(z) = \ln(1 + e^z). \quad (9.5)$$

For Module 2_CPINN, tanh is the best first choice because it is smooth, standard, and stable.

9.4 Hard versus soft boundary enforcement

Boundary conditions can be enforced softly through loss terms or hardwired into the network ansatz. For a rectangular domain with left/right Dirichlet potentials, one can write

$$\phi_\theta(x, y, \tau) = \phi_{\text{bc}}(x, y, \tau) + d(x, y) \mathcal{N}_\theta(x, y, \tau), \quad (9.6)$$

where ϕ_{bc} satisfies the Dirichlet boundary values and $d(x, y)$ vanishes on the Dirichlet boundaries. For example, if $x \in [0, L_x]$ and both $x = 0$ and $x = L_x$ are Dirichlet boundaries,

$$d(x, y) = x(L_x - x). \quad (9.7)$$

A simple boundary interpolant is

$$\phi_{\text{bc}}(x, y, \tau) = \left(1 - \frac{x}{L_x}\right) V_L(\tau) + \frac{x}{L_x} V_R(\tau). \quad (9.8)$$

Then the neural correction cannot violate the left/right Dirichlet values. This can simplify training.

For the first implementation, soft boundary losses are easier to generalize. Hard boundary enforcement can be added later once the CPINN pipeline works.

10 Sampling strategy

10.1 Ordered-state sampling

Choose ordered states

$$\tau_0, \tau_1, \dots, \tau_{N_\tau}. \quad (10.1)$$

The states should be sorted in the physically meaningful direction: increasing time, increasing dose, increasing defect amplitude, or increasing coupling iteration. Causal weights are meaningless if the slices are shuffled.

For early debugging, use a continuation sequence such as

$$\rho(x, y; \tau_i) = a_i \rho_{\text{target}}(x, y), \quad 0 = a_0 < a_1 < \dots < a_{N_\tau} = 1. \quad (10.2)$$

This creates an easy-to-understand ordered sequence from zero charge to target charge.

10.2 Interior residual points

For each slice τ_i , sample interior points

$$\{\mathbf{x}_{r,j}^{(i)}\}_{j=1}^{N_r} \subset \Omega. \quad (10.3)$$

Options include uniform random sampling, Latin hypercube sampling, Sobol sampling, or FEM-node sampling. For the first version, uniform random sampling is sufficient.

The collocation points are not necessarily FEM mesh nodes. FEM nodes are tied to the discretization of the reference solver. PINN collocation points are points where the continuous residual is evaluated.

10.3 Boundary points

For each slice, sample boundary points on Γ_D and Γ_N :

$$\{\mathbf{x}_{D,j}^{(i)}\}_{j=1}^{N_D} \subset \Gamma_D, \quad \{\mathbf{x}_{N,j}^{(i)}\}_{j=1}^{N_N} \subset \Gamma_N. \quad (10.4)$$

Boundary points may be fixed for the full training run or resampled periodically. Fixed points are easier to debug. Resampling gives better coverage.

10.4 Supervised data points

If FEM reference data are available, sample potential values from the FEM solution:

$$\{\mathbf{x}_{s,j}^{(i)}, \phi_{\text{FEM}}(\mathbf{x}_{s,j}^{(i)}, \tau_i)\}_{j=1}^{N_s}. \quad (10.5)$$

For the first CPINN test, use supervised data at the same ordered slices as the residual slices. Later, one can train with sparse supervised slices and test interpolation across unseen slices.

11 Training algorithm

11.1 Algorithm in mathematical form

The Module 2_CPINN training algorithm is:

1. Choose ordered states $\tau_0 < \tau_1 < \dots < \tau_{N_\tau}$.
2. Generate charge maps $\rho(\mathbf{x}; \tau_i, \boldsymbol{\mu})$ and boundary data for each state.
3. Generate collocation, boundary, and optional supervised data points for each state.

4. Initialize neural-network parameters θ .
5. For each training iteration:
 - (a) Compute $\mathcal{L}_{\text{PDE},i}$, $\mathcal{L}_{\text{D},i}$, $\mathcal{L}_{\text{N},i}$, and $\mathcal{L}_{\text{data},i}$ for all slices.
 - (b) Form dimensionless slice losses $\hat{\mathcal{L}}_{\text{slice},i}$.
 - (c) Compute prefix losses $\hat{S}_{i-1} = \sum_{k=0}^{i-1} \hat{\mathcal{L}}_{\text{slice},k}$.
 - (d) Compute weights $w_i = \exp[-\epsilon_c \text{stopgrad}(\hat{S}_{i-1})]$.
 - (e) Compute $\mathcal{L}_{\text{total}}^{\text{CPINN}} = \frac{1}{N_\tau+1} \sum_i w_i \hat{\mathcal{L}}_{\text{slice},i} + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}$.
 - (f) Update θ using Adam or another optimizer.
6. Monitor $\min_i w_i$, slice residuals, FEM validation error, and electric-field error.
7. Stop when validation errors are acceptable and causal weights have activated across the sequence.

11.2 Annealing the causality parameter

The parameter ϵ_c controls the sharpness of the curriculum. If ϵ_c is too small, then

$$w_i \approx 1 \quad (11.1)$$

for all slices and the method reduces to an ordinary PINN. If ϵ_c is too large, then later slices may remain inactive for too long.

A practical strategy is to train through a sequence

$$\epsilon_c^{(1)} < \epsilon_c^{(2)} < \dots < \epsilon_c^{(M)}. \quad (11.2)$$

For example,

$$\epsilon_c \in \{0.1, 1, 10, 100\} \quad (11.3)$$

may be used after all losses are dimensionless and order-one. The exact values should be treated as hyperparameters, not physical constants.

11.3 Causal stopping criterion

A CPINN-specific diagnostic is

$$w_{\min} = \min_i w_i. \quad (11.4)$$

If $w_{\min}$ remains near zero, later ordered states are not active. If $w_{\min}$ approaches one, all states have become active. A possible stopping criterion is

$$w_{\min} > 1 - \delta_w \quad (11.5)$$

combined with validation error criteria such as

$$\frac{\|\phi_\theta - \phi_{\text{FEM}}\|_{L^2(\Omega)}}{\|\phi_{\text{FEM}}\|_{L^2(\Omega)}} < \delta_\phi, \quad (11.6)$$

and

$$\frac{\|\mathbf{E}_\theta - \mathbf{E}_{\text{FEM}}\|_{L^2(\Omega)}}{\|\mathbf{E}_{\text{FEM}}\|_{L^2(\Omega)}} < \delta_E. \quad (11.7)$$

12 Connection to Module 3 and Module 6

12.1 Using Module 3 defect evolution as the causal source

The most physically meaningful ordered variable for Module 2_CPINN is defect-evolution time from Module 3. Module 3 produces

$$C_j(x, y, t_0), C_j(x, y, t_1), \dots, C_j(x, y, t_N). \quad (12.1)$$

These defect concentrations create charge densities

$$\rho(x, y, t_i) = q \left[p - n + N_D^+ - N_A^- + \sum_j z_j C_j(x, y, t_i) \right]. \quad (12.2)$$

Then Module 2 solves

$$\nabla \cdot (\varepsilon \nabla \phi(x, y, t_i)) = -\rho(x, y, t_i) \quad (12.3)$$

for each t_i .

The causal structure comes from Module 3, not from Poisson electrostatics. Module 3 says defects evolve from earlier times to later times. Module 2 says each time snapshot has a quasi-static electrostatic response.

12.2 Using Module 6 coupling iteration as the ordered variable

In a coupled simulation, one may have iteration index m :

$$\rho^{(m)} \rightarrow \phi^{(m)} \rightarrow \mathbf{E}^{(m)} \rightarrow \text{carrier/thermal/defect update} \rightarrow \rho^{(m+1)}. \quad (12.4)$$

The ordered variable may then be the coupling iteration. A CPINN could be trained to respect the sequence of coupled states. This is more advanced and should be postponed until Modules 2 through 5 are individually validated.

12.3 Recommended project sequence

The recommended project path is:

1. Validate baseline Module 2 FEM on simple analytic cases.
2. Validate ordinary Module 2_PINN on the same cases.
3. Create Module 2_CPINN using artificial charge-amplitude continuation.
4. Compare ordinary PINN versus CPINN on interpolation and extrapolation across charge amplitude.
5. Replace artificial continuation with Module 3 defect snapshots.
6. Only then consider using coupling iteration from Module 6.

13 Diagnostics and validation

13.1 Potential error

Compare predicted and FEM potentials at validation points:

$$\varepsilon_{\phi}^{\text{rel}} = \frac{\left[\sum_j |\phi_{\theta}(\mathbf{x}_j) - \phi_{\text{FEM}}(\mathbf{x}_j)|^2 \right]^{1/2}}{\left[\sum_j |\phi_{\text{FEM}}(\mathbf{x}_j)|^2 \right]^{1/2}}. \quad (13.1)$$

If the denominator is small, use an absolute error or normalize by ϕ_0 .

13.2 Electric-field error

The electric field is often more important than potential for downstream carrier transport. Compute

$$\varepsilon_E^{\text{rel}} = \frac{\left[\sum_j \|\mathbf{E}_{\theta}(\mathbf{x}_j) - \mathbf{E}_{\text{FEM}}(\mathbf{x}_j)\|^2 \right]^{1/2}}{\left[\sum_j \|\mathbf{E}_{\text{FEM}}(\mathbf{x}_j)\|^2 \right]^{1/2}}. \quad (13.2)$$

A network can have visually acceptable potential but poor field accuracy because differentiation amplifies errors. Therefore, field error must be a primary metric.

13.3 Residual-by-slice plot

Plot

$$\hat{\mathcal{L}}_{\text{PDE},0}, \hat{\mathcal{L}}_{\text{PDE},1}, \dots, \hat{\mathcal{L}}_{\text{PDE},N_{\tau}} \quad (13.3)$$

versus ordered state. In an ordinary PINN, later residuals may drop before earlier residuals are accurate. In a successful CPINN, residual reduction should move progressively through the ordered sequence.

13.4 Causal-weight plot

Plot

$$w_0, w_1, \dots, w_{N_\tau} \quad (13.4)$$

throughout training. This is the most distinctive CPINN diagnostic. The weights should begin with early slices active and later slices suppressed. As training progresses, weights should activate toward later slices.

13.5 Gauss-law consistency

A physical diagnostic is the integrated Gauss-law error. For a subdomain $A \subset \Omega$,

$$\int_{\partial A} \varepsilon \nabla \phi \cdot \mathbf{n} d\Gamma + \int_A \rho dA = 0. \quad (13.5)$$

For the network,

$$\mathcal{E}_{\text{Gauss}}(A) = \left| \int_{\partial A} \varepsilon \nabla \phi_\theta \cdot \mathbf{n} d\Gamma + \int_A \rho dA \right|. \quad (13.6)$$

This test checks whether the learned field satisfies the integral physics, not just pointwise residual samples.

14 Recommended first CPINN test cases

14.1 Case 1: zero charge with linear potential

Set

$$\rho = 0, \quad \phi(0, y) = V_L, \quad \phi(L_x, y) = V_R, \quad \partial_y \phi(x, 0) = \partial_y \phi(x, L_y) = 0. \quad (14.1)$$

The exact solution is Eq. (6.9). This should be solved by FEM, ordinary PINN, and CPINN. The CPINN ordered variable can be a voltage ramp:

$$V_R(\tau_i) = a_i V_R^{\text{target}}, \quad 0 \leq a_i \leq 1. \quad (14.2)$$

This case checks boundary handling and automatic differentiation, but it does not strongly test charge physics.

14.2 Case 2: Gaussian charge amplitude continuation

Let

$$\rho(x, y; \tau_i) = a_i \rho_0 \exp \left[-\frac{(x - x_c)^2}{2\sigma_x^2} - \frac{(y - y_c)^2}{2\sigma_y^2} \right], \quad (14.3)$$

with

$$0 = a_0 < a_1 < \dots < a_{N_\tau} = 1. \quad (14.4)$$

Use simple boundary conditions, such as grounded Dirichlet boundaries or left/right bias. This is the best first real CPINN test because the ordered variable has a clear continuation meaning.

14.3 Case 3: defect snapshots from Module 3

Use Module 3 to generate defect concentrations $C_j(x, y, t_i)$ and construct

$$\rho(x, y, t_i) = q \sum_j z_j C_j(x, y, t_i) \quad (14.5)$$

plus any background carrier/dopant charge terms included in the chosen approximation. This is the first project-relevant CPINN case. It tests whether causal ordering across defect-evolution time improves electrostatic surrogate training.

14.4 Case 4: coupled-state snapshots from Module 6

Use stored coupled iterations from Module 6:

$$\rho^{(m)}(x, y), \quad \phi^{(m)}(x, y), \quad \mathbf{E}^{(m)}(x, y). \quad (14.6)$$

This should not be the first test. It is useful only after the individual modules are stable.

15 Common failure modes

15.1 Mistaking CPINN for a new electrostatic model

The CPINN does not change Poisson's equation. It changes the training objective. The physics remains

$$\nabla \cdot (\epsilon \nabla \phi) = -\rho. \quad (15.1)$$

If the note or code implies that Module 2 has causal time evolution by itself, that is incorrect.

15.2 Using unscaled losses in causal exponentials

The causal weight contains an exponential. If losses are not normalized, the model may produce

$$w_i \approx 0 \quad (15.2)$$

for almost every later slice. This is not a deep physical result; it is numerical underflow or bad scaling.

15.3 Overweighting supervised data

If λ_s is too large, the network may fit FEM values but ignore the residual. If λ_r is too large, the network may reduce residual at collocation points while fitting boundary/data poorly. CPINN does not remove the need for loss balancing.

15.4 Using too many ordered slices too early

A large number of slices increases cost and can make the activation front difficult to monitor. Begin with 5 to 10 ordered states. Increase after the basic behavior is validated.

15.5 Poor charge-parameter representation

If the network input does not encode the global charge distribution, it cannot learn a general Poisson operator. For a parameterized Gaussian charge, parameters are sufficient. For arbitrary charge maps, a simple MLP input is not enough unless one trains a separate network for a fixed sequence.

16 What should be implemented first

The first Module 2_CPINN implementation should be intentionally modest:

1. Use a fixed rectangular domain.
2. Use constant silicon permittivity.
3. Use smooth tanh MLP.
4. Use dimensionless coordinates, potential, and charge density.
5. Use artificial Gaussian charge-amplitude continuation.
6. Use soft Dirichlet and Neumann boundary losses.

7. Use full-slice causal weighting first.
8. Compare against ordinary Module 2_PINN and FEM.
9. Plot residual by slice and causal weights.

After this works, add:

1. hard boundary enforcement;
2. residual-only causal weighting;
3. Module 3 defect snapshots;
4. optional finite-difference ordered-consistency residual;
5. interpolation tests on unseen dose/time slices.

17 Minimal mathematical specification for the code

The code-level mathematical model should be the following.

17.1 Inputs

Use normalized inputs

$$\mathbf{z} = \begin{bmatrix} \hat{x} & \hat{y} & \hat{\tau} & \hat{\boldsymbol{\mu}}^T \end{bmatrix}^T. \quad (17.1)$$

The neural network returns

$$\hat{\phi}_\theta = \mathcal{N}_\theta(\mathbf{z}). \quad (17.2)$$

17.2 Residual

For each slice,

$$\hat{\mathcal{R}}_P^{(i)} = \frac{\partial^2 \hat{\phi}_\theta}{\partial \hat{x}^2} + \frac{\partial^2 \hat{\phi}_\theta}{\partial \hat{y}^2} + \beta \hat{\rho}^{(i)}. \quad (17.3)$$

17.3 Slice losses

Use

$$\hat{\mathcal{L}}_{\text{slice},i} = \lambda_r \hat{\mathcal{L}}_{\text{PDE},i} + \lambda_D \hat{\mathcal{L}}_{\text{D},i} + \lambda_N \hat{\mathcal{L}}_{\text{N},i} + \lambda_s \hat{\mathcal{L}}_{\text{data},i}. \quad (17.4)$$

17.4 Causal weights

Use

$$\hat{S}_{i-1} = \sum_{k=0}^{i-1} \hat{\mathcal{L}}_{\text{slice},k}, \quad w_i = \exp \left[-\epsilon_c \text{stopgrad}(\hat{S}_{i-1}) \right]. \quad (17.5)$$

17.5 Total loss

Use

$$\hat{\mathcal{L}}_{\text{total}}^{\text{CPINN}} = \frac{1}{N_\tau + 1} \sum_{i=0}^{N_\tau} w_i \hat{\mathcal{L}}_{\text{slice},i} + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}. \quad (17.6)$$

This is the primary target equation set for `main_module2_cpinn_electrostatics.m` or the equivalent Module 2 CPINN MATLAB driver.

18 Summary

The Module 2_CPINN extension should be understood as a causality-respecting training strategy for ordered electrostatic solution families. The governing physics remains Poisson electrostatics. The new ingredient is an ordered loss weighting:

$$w_i = \exp \left[-\epsilon_c \sum_{k < i} \hat{\mathcal{L}}_{\text{slice},k} \right], \quad (18.1)$$

which suppresses later states until earlier states have been learned. For genuinely time-dependent PDEs, this directly respects temporal causality. For Module 2, which is elliptic and quasi-static, the causal coordinate must come from outside the Poisson equation: dose, damage time, defect-evolution snapshots, voltage continuation, or multiphysics coupling iteration.

The scientifically correct first implementation is not to claim that electrostatic Poisson physics evolves causally. Instead, train a neural surrogate over an ordered sequence of quasi-static Poisson problems and test whether causal residual weighting improves convergence, stability, and interpolation compared with the ordinary Module 2_PINN.

References

- [1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686-707, 2019.

- [2] S. Wang, S. Sankaran, and P. Perdikaris, “Respecting causality is all you need for training physics-informed neural networks,” arXiv:2203.07404, 2022.
- [3] S. Wang, S. Sankaran, and P. Perdikaris, “Respecting causality for training physics-informed neural networks,” *Computer Methods in Applied Mechanics and Engineering*, vol. 421, 116813, 2024.
- [4] S. Wang, S. Sankaran, H. Wang, and P. Perdikaris, “An expert’s guide to training physics-informed neural networks,” arXiv:2308.08468, 2023.
- [5] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, “Physics-informed machine learning,” *Nature Reviews Physics*, vol. 3, pp. 422-440, 2021.
- [6] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, SIAM, 2008.